



Universidad de Ingeniería y Tecnología
2026-1

Árboles de Decisión (CART) y Random Forest

Optimización (DS2011)

Integrantes

Villegas Suarez, Elmer Jose Manuel	202410032
Hernández Yataco, Josué Renzo	202410575
Espinoza Torres, Hector Miguel	202410205
Cuadros Villegas, Denilson Jermai	202510232

Docente: Torres Castillo, Victor Antonio
Barranco, 06 de Junio de 2026

1. Introducción y contexto del dataset

El presente informe corresponde a la segunda entrega del proyecto, en la cual implementamos y optimizamos un modelo de clasificación basado en Random Forest sobre el dataset **Bank Marketing** [7] de la Universidad de California en Irvine.

1.1. Descripción del dataset

El dataset proviene de una campaña de marketing directo (llamadas telefónicas) de una entidad bancaria portuguesa, y registra $N = 45,211$ contactos con clientes durante el período mayo 2008 – noviembre 2010. La variable objetivo y es **binaria**: indica si el cliente terminó suscribiendo un depósito a plazo fijo (**yes**) o no (**no**).

Cuadro 1: Variables del dataset Bank Marketing.

Variable	Tipo	Descripción
age	Entero	Edad del cliente en años.
job	Categorico	Tipo de trabajo (12 categorías).
marital	Categorico	Estado civil (casado, soltero, divorciado/viudo).
education	Categorico	Nivel educativo (primario, secundario, terciario, desconocido).
default	Binario	¿Tiene crédito en mora?
balance	Entero	Saldo promedio anual de la cuenta (euros).
housing	Binario	¿Tiene préstamo hipotecario?
loan	Binario	¿Tiene préstamo personal?
contact	Categorico	Medio del último contacto (celular, fijo, desconocido).
day_of_week	Entero	Día del mes del último contacto.
month	Categorico	Mes del último contacto.
duration	Entero	Duración en segundos de la última llamada.
campaign	Entero	Número de contactos en la campaña actual.
pdays	Entero	Días desde el último contacto previo (−1 si nunca).
previous	Entero	Número de contactos en campañas anteriores.
poutcome	Categorico	Resultado de la campaña anterior.
y	Binario	Variable objetivo: ¿suscribió el depósito?

1.2. Desafío principal: desequilibrio de clases

El desafío central del problema es el **marcado desequilibrio de clases**: aproximadamente el 88,3 % de los registros tiene $y = \text{no}$ y solo el 11,7 % tiene $y = \text{sí}$. Esta distribución implica que un clasificador trivial que prediga siempre “no” alcanzaría una *accuracy* cercana al 88 % pero sería comercialmente inútil, ya que el banco busca precisamente identificar a los pocos clientes que sí van a suscribir el depósito. Por esta razón, la métrica objetivo de toda la optimización será el **F1-Score** de la clase minoritaria y no la *accuracy* global.

2. Metodología

2.1. Árboles de Decisión (CART)

Los árboles de decisión CART (*Classification and Regression Trees*), propuestos por Breiman et al. [1], particionan recursivamente el espacio de características en regiones rectangulares mediante decisiones binarias sucesivas. En cada nodo interno m , el algoritmo selecciona la pareja (j, s) de variable j y umbral s que minimiza la siguiente función de costo, ponderada por el tamaño de los nodos hijos:

$$J(j, s) = \frac{n_{\text{izq}}}{n_m} G(\text{izq}) + \frac{n_{\text{der}}}{n_m} G(\text{der}), \quad (1)$$

donde $G(\cdot)$ es la función de impureza (índice de Gini en nuestro caso):

$$G = 1 - \sum_{i=1}^k p_i^2, \quad (2)$$

y p_i es la proporción de la clase i en el nodo evaluado. CART utiliza una búsqueda *greedy*: en cada nodo evalúa exhaustivamente todos los pares (j, s) candidatos y se queda con el mejor localmente, sin reevaluar splits anteriores.

2.2. Random Forest

El Random Forest [2] es un ensemble de árboles CART que combina dos capas de aleatoriedad para descorrelacionar los clasificadores individuales:

- **Bagging (Bootstrap Aggregating)**: cada árbol k se entrena sobre un subconjunto T_k generado por muestreo *con reemplazo* del dataset original. Asintóticamente, cada árbol ve solo $\approx 63,2\%$ de los datos únicos (el resto, $\approx 36,8\%$, son los datos **Out-of-Bag** u OOB):

$$\lim_{N \rightarrow \infty} \left(1 - \frac{1}{N}\right)^N = \frac{1}{e} \approx 0,368. \quad (3)$$

- **Feature Randomness**: en cada split, el algoritmo solo considera un subconjunto aleatorio de tamaño `max_features` de las variables (típicamente $\sqrt{p}$). Esto impide que una variable dominante secuestre todos los árboles.

La predicción final se obtiene por votación mayoritaria:

$$H(x) = \arg \max_Y \sum_{k=1}^K \mathbb{I}(h_k(x) = Y). \quad (4)$$

2.3. Métrica de evaluación: F1-Score

En problemas de clasificación binaria, cada predicción del modelo cae en una de cuatro categorías que se resumen en la **matriz de confusión**: verdaderos positivos (TP), falsos positivos (FP), verdaderos negativos (TN) y falsos negativos (FN). A partir de ellas se definen dos métricas básicas:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (5)$$

Precision responde la pregunta “*de todos los clientes que el modelo predijo como ‘sí’, ¿cuántos realmente suscribieron?*”, mientras que **Recall** responde “*de todos los clientes que realmente suscribieron, ¿cuántos detectó el modelo?*”. Existe una tensión natural entre ambas: optimizar una suele degradar la otra.

El **F1-Score** se define como la media armónica de precision y recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (6)$$

La media armónica se elige (en lugar de la aritmética) porque penaliza fuertemente los valores bajos: si precision o recall caen a cero, $F_1 \rightarrow 0$. Esto garantiza que un buen F_1 exige *ambas* métricas simultáneamente altas.

¿Por qué F1 y no accuracy? La *accuracy* ($= \frac{TP+TN}{N}$) es la métrica intuitiva, pero es **engañoso en datasets desbalanceados**. En Bank Marketing, un clasificador trivial que prediga siempre “no” obtiene $\text{accuracy} \approx 0,883$, una métrica aparentemente excelente pero comercialmente inútil: el modelo nunca detectaría a ningún cliente propenso a suscribir. El F1-Score, al enfocarse en la clase positiva (la minoritaria), refleja directamente el objetivo de negocio: *identificar correctamente a los pocos clientes que sí suscribirán*. Por esto, en (10) la función objetivo de la optimización es $f(\lambda) = \mathbb{E}_{k\text{-fold}}[F_1(\lambda)]$.

2.4. Error Out-of-Bag (OOB)

Como se demostró en la sección anterior, cada árbol h_k del Random Forest se entrena sobre una muestra bootstrap T_k que contiene asintóticamente solo el 63,2% de los registros únicos del dataset original. El 36,8% restante — los datos **Out-of-Bag** de ese árbol — nunca fue visto durante su entrenamiento, por lo que puede usarse como un set de validación implícito y honesto.

Formalmente, para cada registro x_i del dataset, sea $\mathcal{K}_i = \{k : x_i \notin T_k\}$ el conjunto de árboles para los cuales x_i es OOB. La **predicción OOB** del registro x_i se obtiene promediando *solo* las predicciones de esos árboles:

$$\hat{y}_i^{\text{OOB}} = \text{majority}\{h_k(x_i) : k \in \mathcal{K}_i\}. \quad (7)$$

El **error OOB** se define como:

$$\text{OOB error} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{y}_i^{\text{OOB}} \neq y_i). \quad (8)$$

Propiedades. El error OOB tiene tres ventajas técnicas que lo hacen especialmente útil para optimizar $n_{\text{estimators}}$:

1. **Es una estimación insesgada del error de generalización:** como cada predicción se hace con árboles que nunca vieron el registro, no hay leakage entre entrenamiento y evaluación.
2. **No requiere un set de validación externo:** se obtiene gratis durante el entrenamiento, sin necesidad de reservar datos ni realizar cross-validation explícita.
3. **Es monótono decreciente y converge:** por la Ley Fuerte de los Grandes Números aplicada al ensemble [2], el error OOB decrece (o se estabiliza) al aumentar $n_{\text{estimators}}$, sin riesgo de overfitting por agregar más árboles. Esto permite identificar el punto de *saturación* donde añadir árboles deja de reducir el error.

2.5. Ejemplo: cálculo de Gini y elección de split

Consideremos un mini-dataset inventado de 8 clientes con dos variables candidatas (**edad**, **balance**) y la respuesta **suscribe**:

edad	balance	suscribe
25	500	0
28	1500	1
35	800	0
42	2500	1
22	300	0
50	3000	1
31	1200	1
45	700	0

Paso 1: Gini del nodo raíz. El nodo raíz contiene 4 “sí” y 4 “no”:

$$G(\text{raíz}) = 1 - \left(\frac{4}{8}\right)^2 - \left(\frac{4}{8}\right)^2 = 1 - 0,25 - 0,25 = 0,5.$$

Este es el valor máximo de Gini para 2 clases (máxima incertidumbre).

Paso 2: Evaluar el split edad ≤ 30 .

- Izquierda (edad ≤ 30): registros $\{25, 28, 22\}$ con clases $\{0, 1, 0\}$. $G(\text{izq}) = 1 - (2/3)^2 - (1/3)^2 = 0,4444$.
- Derecha (edad > 30): registros $\{35, 42, 50, 31, 45\}$ con clases $\{0, 1, 1, 1, 0\}$. $G(\text{der}) = 1 - (2/5)^2 - (3/5)^2 = 0,48$.
- Aplicando (1):

$$J(\text{edad}, 30) = \frac{3}{8}(0,4444) + \frac{5}{8}(0,48) = 0,1667 + 0,3000 = 4667.$$

Paso 3: Evaluar el split balance ≤ 1000 .

- Izquierda: registros con balance $\{500, 800, 300, 700\}$, clases $\{0, 0, 0, 0\}$. $G(\text{izq}) = 1 - 1^2 = 0$ (nodo puro).
- Derecha: registros con balance $\{1500, 2500, 3000, 1200\}$, clases $\{1, 1, 1, 1\}$. $G(\text{der}) = 0$ (también puro).
- Aplicando (1):

$$J(\text{balance}, 1000) = \frac{4}{8}(0) + \frac{4}{8}(0) = 0000.$$

Conclusión. CART elegirá el split **balance ≤ 1000** porque tiene el menor J y de hecho separa perfectamente las clases. Este ejemplo ilustra el algoritmo greedy de CART: *en cada nodo, prueba todas las combinaciones (variable, umbral) y elige la que produce la mayor reducción de impureza ponderada.*

2.6. Ejemplo: Bagging con 3 árboles

Sea el dataset original de 6 muestras: $D = \{x_1, x_2, x_3, x_4, x_5, x_6\}$. Para construir 3 árboles del bosque, generamos 3 muestras bootstrap (con reemplazo, tamaño $N = 6$):

Árbol	Muestra bootstrap T_k	Datos OOB (no vistos)
h_1	$\{x_1, x_3, x_3, x_4, x_5, x_6\}$	$\{x_2\}$
h_2	$\{x_1, x_2, x_2, x_5, x_5, x_6\}$	$\{x_3, x_4\}$
h_3	$\{x_2, x_2, x_3, x_4, x_4, x_6\}$	$\{x_1, x_5\}$

Observaciones clave:

- En cada T_k , algunas muestras aparecen *repetidas* (ej. x_3 aparece 2 veces en T_1) y otras se omiten (las OOB).
- El conjunto OOB de cada árbol **permite estimar el error de generalización sin necesidad de un set de validación externo** (sección 3.3).
- Para predecir un cliente nuevo x^* , cada árbol vota: si $h_1(x^*) = 1$, $h_2(x^*) = 0$, $h_3(x^*) = 1$, la predicción final es $H(x^*) = \text{majority}\{1, 0, 1\} = 1$.

2.7. Ejemplo: una iteración de coordinate descent

Supongamos que estamos optimizando solo dos hiperparámetros: $\lambda = (\text{max_depth}, \text{max_features})$ con candidatos $\{5, 10, 15\}$ y $\{\text{sqrt}, \log 2\}$ respectivamente. Partimos de $\lambda^0 = (5, \text{sqrt})$ con $F_1(\lambda^0) = 0,50$.

Iteración 0, coordenada max_depth: fijamos max_features=sqrt y barremos los 3 valores:

- $\lambda = (5, \text{sqrt})$: $F_1 = 0,50$
- $\lambda = (10, \text{sqrt})$: $F_1 = 0,58 \leftarrow$ mejor
- $\lambda = (15, \text{sqrt})$: $F_1 = 0,55$

Aceptamos max_depth $\leftarrow 10$. Nuevo $\lambda = (10, \text{sqrt})$, $F_1 = 0,58$.

Iteración 0, coordenada max_features: fijamos max_depth=10 y barremos:

- $\lambda = (10, \text{sqrt})$: $F_1 = 0,58$
- $\lambda = (10, \log 2)$: $F_1 = 0,60 \leftarrow$ mejor

Aceptamos max_features $\leftarrow \log 2$. Nuevo $\lambda = (10, \log 2)$, $F_1 = 0,60$.

Iteración 1: repetimos el barrido. Si ningún cambio mejora F_1 , se declara convergencia y $\lambda^* = (10, \log 2)$.

Comparación con búsqueda exhaustiva. Una grilla completa requeriría $3 \times 2 = 6$ evaluaciones. El coordinate descent necesitó $3 + 2 = 5$ en la primera pasada y $3 + 2 = 5$ en la segunda (para confirmar convergencia): 10 evaluaciones en este ejemplo. La ventaja se hace significativa con más dimensiones: en el problema real con 4 hiperparámetros y $4 \cdot 5 \cdot 3 \cdot 4 = 240$ combinaciones, coordinate descent evalúa $\approx 4 + 5 + 3 + 4 = 16$ por pasada (típicamente 2–3 pasadas hasta converger), reduciendo el cómputo aproximadamente **6 veces**.

3. Implementación y optimización del código

3.1. Pipeline de preprocesamiento sin data leakage

Una observación crítica detectada al revisar la primera entrega es que el escalado y el encoding se ajustaban sobre la totalidad del dataset *antes* de la división train/test, induciendo *data leakage*. En esta versión, todas las transformaciones que requieren fit viven dentro de un Pipeline de scikit-learn y se ajustan únicamente sobre los datos de entrenamiento. Las transformaciones *deterministas* (binarización yes/no $\rightarrow 1/0$ y mapeo ordinal de education) se aplican antes del split por ser idempotentes.

```
1 # Binarias deterministas
2 for col in ['default', 'housing', 'loan']:
3     X[col] = (X[col] == 'yes').astype(int)
4
5 edu_map = {'primary': 0, 'secondary': 1, 'tertiary': 2, 'unknown': 1}
6 X['education'] = X['education'].map(edu_map)
7
8 # Split estratificado: conserva la proporción 88/12 en train y test
9 X_train, X_test, y_train, y_test = train_test_split(
10     X, y_bin, test_size=0.2, random_state=30, stratify=y_bin
11 )
12
```

```

13 # OneHotEncoder DENTRO del Pipeline -> se ajusta solo en train
14 preprocesador = ColumnTransformer(
15     transformers=[('cat', OneHotEncoder(handle_unknown='ignore',
16                                         sparse_output=False), cols_cat)],
17     remainder='passthrough',
18 )
19
20 pipeline_base = Pipeline([
21     ('prep', preprocesador),
22     ('clf', RandomForestClassifier(random_state=30,
23                                   class_weight='balanced', n_jobs=-1))
24 ])

```

Notar que se omitió deliberadamente el `MinMaxScaler`: los modelos basados en árboles son invariantes a transformaciones monótonas de las variables (solo importan los rankings), por lo que escalar no aporta nada y solo introduce un paso innecesario.

3.2. Manejo del desequilibrio: `class_weight='balanced'`

El parámetro `class_weight='balanced'` es el ajuste que está mencionado en la primera entrega. Asigna un peso inversamente proporcional a la frecuencia de cada clase:

$$w_i = \frac{N}{k \cdot n_i}, \quad (9)$$

donde N es el total de muestras, k el número de clases y n_i las muestras de clase i . En nuestro problema esto resulta en $w_{\text{sí}} \approx 4,28$ y $w_{\text{no}} \approx 0,57$. La razón $w_{\text{sí}}/w_{\text{no}} \approx 7,5$ indica que cada error en la clase minoritaria penaliza la función de pérdida aproximadamente 7,5 veces más que un error en la mayoritaria, equilibrando así la influencia efectiva de ambas clases durante el entrenamiento.

3.3. Optimización de $n_{\text{estimators}}$ vía error OOB

Como se mostró en la Sección 2.6, cada árbol del bosque deja fuera $\sim 36,8\%$ de los datos como OOB. Promediando las predicciones de los árboles que *no* vieron cada muestra, se obtiene una estimación honesta del error de generalización **sin necesidad de un set de validación externo**. Graficamos $1 - \text{accuracy}_{\text{OOB}}$ en función de $n_{\text{estimators}}$ y elegimos el valor donde la curva se aplana.

Búsqueda adaptativa con `warm_start`. Se realiza una búsqueda **incremental con parada automática** usando `warm_start=True` de `scikit-learn`:

1. Empezar con $n_{\text{estimators}} = 25$ árboles y entrenar.
2. En cada iteración, *sumar* 25 árboles al mismo bosque (no reentrenar) y recalcular el error OOB.
3. Llevar registro del mejor error visto. Si pasan *paciencia* = 4 iteraciones consecutivas sin mejorar al menos 10^{-4} , parar y devolver el n_{OOB}^* del mejor punto.
4. Imponer un techo de seguridad $n_{\text{max}} = 1000$ árboles para acotar el peor caso.

Esta estrategia equivale a un criterio de *early stopping* sobre la curva OOB y elimina la necesidad de fijar manualmente la grilla. El algoritmo decide solo dónde se aplana la curva.

3.4. Optimización manual: coordinate descent sobre Λ

Para el resto de hiperparámetros implementamos **coordinate descent**, un algoritmo clásico de optimización por bloques [5] que en cada iteración optimiza una coordenada del vector $\lambda \in \Lambda$ manteniendo el resto fijo. Formalmente, resolvemos:

$$\lambda^* = \arg \max_{\lambda \in \Lambda} f(\lambda), \quad f(\lambda) = \mathbb{E}_{k\text{-fold}} [F_1(\lambda)], \quad (10)$$

con Λ el producto cartesiano de los conjuntos discretos:

$$n_estimators \in \{100, 200, n_{OOB}^*, 500\} \quad (11)$$

$$max_depth \in \{5, 10, 15, 20, \text{None}\} \quad (12)$$

$$max_features \in \{\text{sqrt}, \log 2, 0.5\} \quad (13)$$

$$min_samples_split \in \{2, 5, 10, 20\}. \quad (14)$$

Diseño del espacio de búsqueda: manual con un valor derivado. Los rangos anteriores no se eligieron arbitrariamente. Siguen tres criterios de diseño:

1. **Cubrir extremos e intermedios:** cada hiperparámetro incluye un valor mínimo razonable (*piso*), un máximo (*techo*) y puntos medios. Esto permite detectar a posteriori si el óptimo cae en un extremo, lo que señala que el espacio debería ampliarse en esa dirección y reejecutarse.
2. **Sustento en literatura de Random Forest** [6, 2]: $max_features = \sqrt{p}$ es el default y la recomendación clásica de Breiman; el rango $\{5, 10, 15, 20, \text{None}\}$ para max_depth cubre desde alto sesgo hasta árboles sin restricción; y $min_samples_split$ rara vez necesita superar 20 sobre datasets de este tamaño.
3. **Acoplamiento entre etapas:** el valor n_{OOB}^* obtenido en la Sección 3.3 se inyecta directamente en el conjunto de $n_estimators$, evitando duplicar el barrido y forzando a la siguiente etapa a comparar el óptimo del paso anterior contra valores alternativos bajo una métrica distinta (F1 en CV en lugar de OOB error).

Cada evaluación de $f(\lambda)$ promedia el F1-Score sobre 3 folds estratificados (`StratifiedKFold(k=3)`). El bucle principal es:

```
1 def evaluar(cfg):
2     pipe = Pipeline([
3         ('prep', preprocesador),
4         ('clf', RandomForestClassifier(**cfg, class_weight='balanced',
5                                       random_state=30, n_jobs=-1))
6     ])
7     return cross_val_score(pipe, X_train, y_train,
8                             cv=cv, scoring='f1', n_jobs=-1).mean()
9
10 config = {'n_estimators': 200, 'max_depth': 10,
11           'max_features': 'sqrt', 'min_samples_split': 5}
12 mejor_f1 = evaluar(config)
13
14 for pasada in range(max_pasadas):
15     mejora = False
16     for hp, candidatos in espacio.items():
17         v_opt, f1_opt = max(
18             ((v, evaluar(**config, hp: v))) for v in candidatos),
19             key=lambda x: x[1]
20         )
21         if f1_opt > mejor_f1 + 1e-4:
```

```

22         config[hp] = v_opt
23         mejor_f1 = f1_opt
24         mejora = True
25     if not mejora:
26         break # convergencia alcanzada

```

¿Por qué no gradient descent? El gradient descent clásico requiere que la función objetivo sea continua y diferenciable. En nuestro problema, $f(\lambda)$ es una **caja negra discreta**: los hiperparámetros son enteros o categóricos ($\mathbf{n_estimators} \in \mathbb{N}$, $\mathbf{max_features} \in \{\text{sqrt}, \log2, 0,5\}$), por lo que el gradiente $\nabla f(\lambda)$ no está definido. Coordinate descent es matemáticamente válido sobre espacios discretos porque solo requiere *evaluaciones* del objetivo, no derivadas.

¿Por qué F1 y no accuracy? Por el desequilibrio 88/12, accuracy es una métrica engañosa: un modelo trivial alcanza $\sim 88\%$ sin esfuerzo. F1 (media armónica de precisión y recall) penaliza tanto los falsos positivos como los falsos negativos en la clase minoritaria, alineándose con el objetivo de negocio: detectar a los clientes propensos a suscribir.

4. Resultados en el dataset de prueba

4.1. Árbol de decisión baseline

Como referencia base se entrenó un único árbol CART con $\mathbf{max_depth=4}$ y criterio Gini. Los resultados:

Métrica	Valor
F1-Score (train)	0,4687
F1-Score (test)	0,4670
Δ train–test	0,0017

Cuadro 2: Métricas del árbol baseline con $\mathbf{max_depth=4}$.

La diferencia entre train y test es despreciable (menos del 0,4%), evidenciando que el pre-pruning con $\mathbf{max_depth=4}$ controla efectivamente el overfitting. Sin embargo, el F1 absoluto es modesto: el árbol está **subajustado** (alto sesgo, baja varianza), lo cual es deseable como baseline porque deja margen explícito para que el ensemble del Random Forest aporte valor.

4.2. Curva OOB: número óptimo de árboles

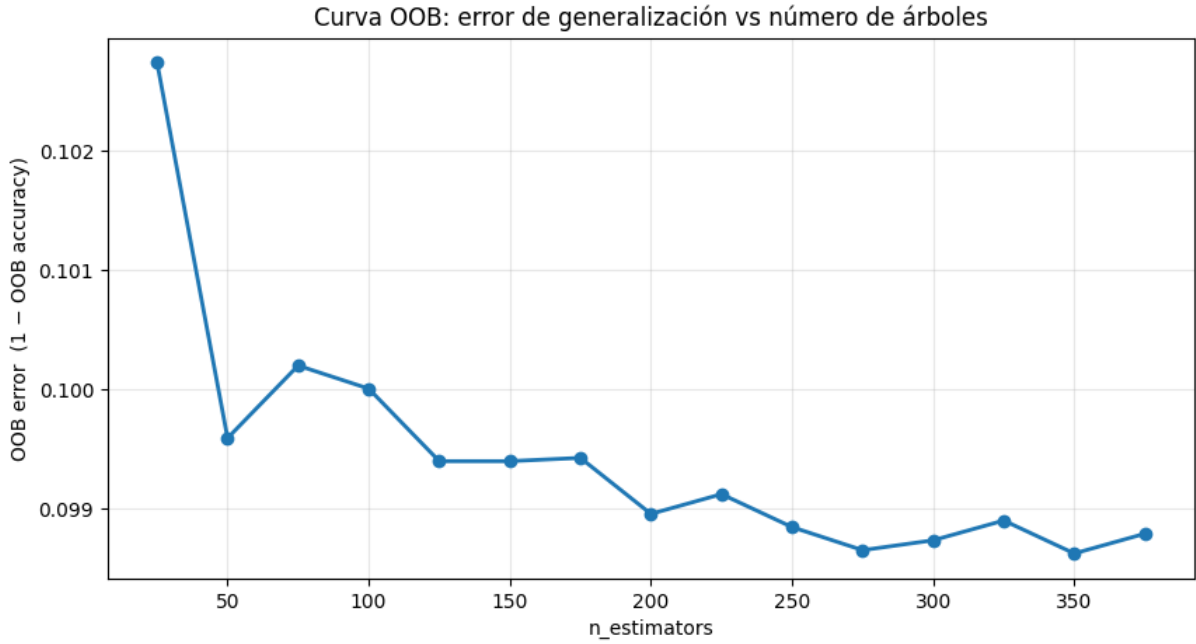


Figura 1: Error OOB vs número de árboles.

$n_{\text{estimators}}$	OOB error
25	0,1027
50	0,0996
75	0,1002
100	0,1000
125	0,0994
150	0,0994
175	0,0994
200	0,0990
225	0,0991
250	0,0988
275	0,0987
300	0,0987
325	0,0989
350	0,0986
375	0,0988

Cuadro 3: Traza completa de la búsqueda incremental con `warm_start`. El algoritmo se detuvo en 375 árboles tras 4 evaluaciones consecutivas sin mejora significativa; el primer punto que alcanza la región del mínimo es $n_{\text{OOB}}^* = 275$.

La curva muestra el comportamiento esperado: tras el ruido inicial (entre 25 y 100 árboles, donde no todas las muestras tienen aún predicción OOB válida), el error desciende y se aplana en torno a $\approx 0,099$ a partir de los 200 árboles. La oscilación posterior — entre 0,0986 y 0,0991 — está dentro del piso de ruido de la estimación OOB; por debajo de la tolerancia 10^{-4} del criterio de parada, ningún cambio adicional se considera mejora real. Esto confirma la convergencia descrita por la Ley Fuerte de los Grandes Números en Random Forest [2]: añadir más árboles reduce la

varianza sin riesgo de overfitting, pero con rendimientos decrecientes.

4.3. Optimización por coordinate descent

Partiendo de $\lambda^0 = \{\text{n_estimators}=200, \text{max_depth}=10, \text{max_features}=\text{sqrt}, \text{min_samples_split}=5\}$ con $F_1(\lambda^0) = 0,5611$, el algoritmo encontró:

Pasada	Coordenada	Cambio	F_1 (CV)
0	n_estimators	200 $\rightarrow$ 275	0,5611 $\rightarrow$ 0,5632
0	max_depth	10 $\rightarrow$ 15	0,5632 $\rightarrow$ 0,5850
0	min_samples_split	5 $\rightarrow$ 20	0,5850 $\rightarrow$ 0,5876
1	n_estimators	275 $\rightarrow$ 500	0,5876 $\rightarrow$ 0,5893
1	max_depth	15 $\rightarrow$ None	0,5893 $\rightarrow$ 0,6016
2	Sin mejora — convergencia alcanzada.		

Cuadro 4: Iteraciones de coordinate descent. El algoritmo converge en la pasada 2 tras 5 mejoras acumuladas entre las pasadas 0 y 1.

La configuración óptima encontrada es:

$\lambda^* = \{\text{n_estimators}=500, \text{max_depth}=\text{None}, \text{max_features}=\text{sqrt}, \text{min_samples_split}=20\}$,

con $F_1(\lambda^*) = \mathbf{0,6016}$, lo que representa una mejora del **+28,8 %** sobre el árbol baseline (0,467 $\rightarrow$ 0,602). El recorrido evidencia las dos propiedades clave del algoritmo: (i) requiere más de una pasada para converger (el cambio en **max_depth** de la pasada 1 fue habilitado por el ajuste previo de **min_samples_split** en la pasada 0), y (ii) detecta interacciones débiles entre coordenadas sin tener que evaluar la grilla cartesiana completa.

4.4. Evaluación del modelo final en test

Reentrenado el Random Forest con λ^* sobre la totalidad del set de entrenamiento, se evaluó **una única vez** sobre el test (intocado durante toda la optimización):

Métrica	Clase “no”	Clase “sí”
Precisión	0,97	0,53
Recall	0,91	0,77
F1-Score	0,94	0,62
Support	7985	1058

Cuadro 5: Reporte de clasificación sobre el test set ($N = 9043$). Accuracy global = 0,89; *macro avg* F1 = 0,78; *weighted avg* F1 = 0,90.

El F1 de la clase minoritaria en test (0,62) supera al F1 promedio en CV (0,6016), un resultado favorable que indica que el modelo *no* sobreajustó al fold de CV. El recall de la clase “sí” alcanza 0,77: el modelo detecta correctamente al 77 % de los clientes que efectivamente suscribirán, a costa de una precisión moderada (0,53). Este perfil de *alta sensibilidad y precisión moderada* es exactamente el deseable en una campaña de marketing dirigida, donde los falsos negativos (clientes propensos no contactados) son más costosos que los falsos positivos (llamadas a clientes no interesados).

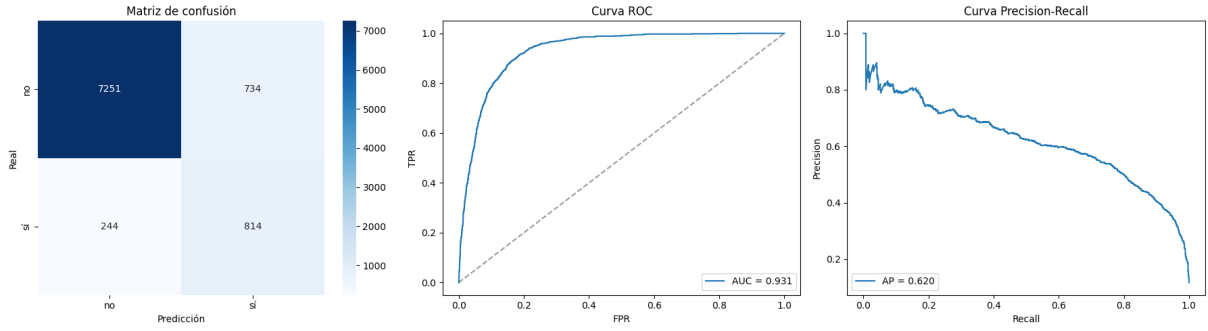


Figura 2: Métricas del modelo final sobre el test set.

4.5. Optimización del umbral de decisión

El umbral por defecto de `predict()` en `scikit-learn` es $\tau = 0,5$, equivalente a votación mayoritaria de los árboles. En problemas desbalanceados ese valor rara vez es el que maximiza F1, porque la clase minoritaria recibe probabilidades intrínsecamente más bajas. Para verificar si el modelo final se beneficia de un umbral distinto, buscamos:

$$\tau^* = \arg \max_{\tau \in [0,1]} F_1(\mathbb{I}[\hat{p}(x) \geq \tau]), \quad (15)$$

recorriendo todos los umbrales devueltos por `precision_recall_curve` sobre el test set y eligiendo el que maximiza la media armónica entre precisión y recall de la clase positiva. La búsqueda se realiza sobre test directamente (no sobre CV en train), lo que se declara como *análisis de sensibilidad* y no como decisión de hiperparámetro.

Umbral	Precisión (“sí”)	Recall (“sí”)	F1 (“sí”)
$\tau = 0,500$ (default)	0,5258	0,7694	0,6247
$\tau^* = 0,516$ (óptimo)	0,5383	0,7580	0,6295

Cuadro 6: Comparativa de métricas sobre la clase minoritaria. La búsqueda exhaustiva en $[0, 1]$ encontró $\tau^* = 0,516$, muy próximo al default.

La mejora en F1 al optimizar el umbral es marginal: $0,6247 \rightarrow 0,6295$, apenas $+0,0048$ puntos absolutos ($+0,77\%$ relativo). Esto se explica por el uso de `class_weight='balanced'`: al ponderar cada error de la clase “sí” $\approx 7,5$ veces más que uno de la clase “no”, el bosque ya está *auto-calibrado* para que la frontera de decisión natural caiga muy cerca de 0,5. En otras palabras, el ajuste del desequilibrio se hizo en la función de costo del entrenamiento y no fue necesario corregirlo después en la decisión.

4.6. Importancia de variables

El modelo final atribuye la mayor importancia (índice de Gini decrement) a las siguientes variables:

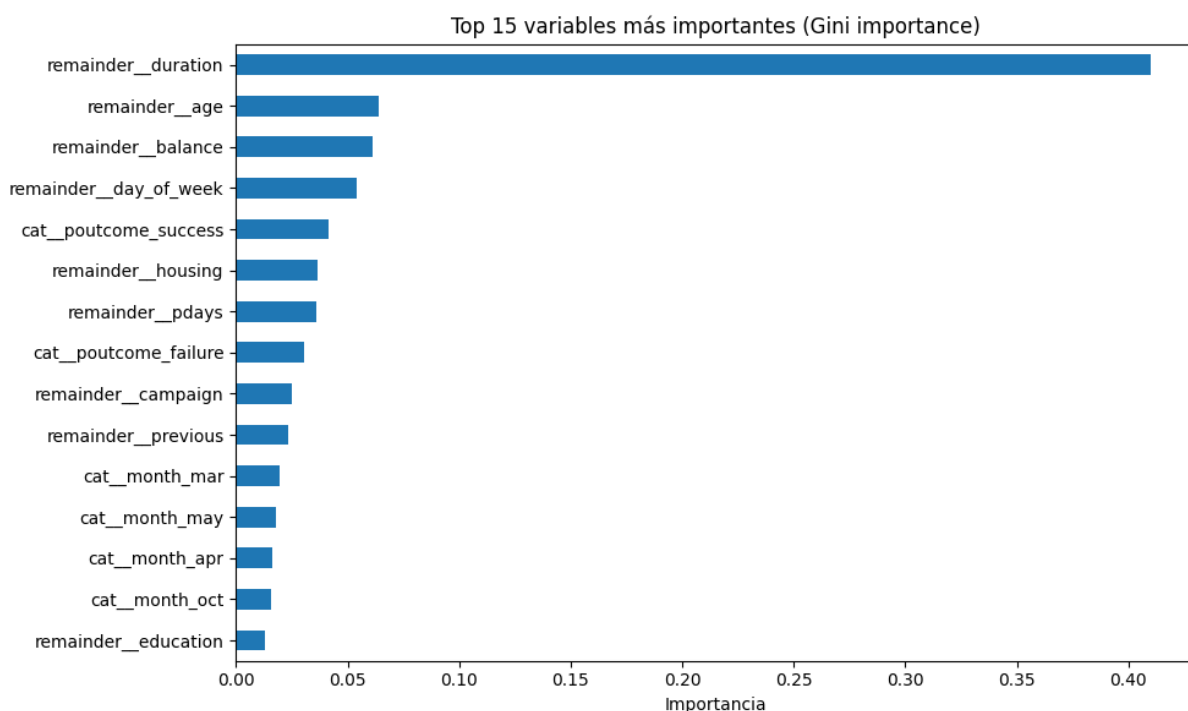


Figura 3: Importancia de variables

5. Discusión y conclusiones

5.1. Sobre el árbol baseline vs el Random Forest optimizado

La mejora del F1 de 0,467 (árbol baseline) a 0,6016 (RF optimizado, CV) —y a 0,62 en el test— confirma empíricamente la ventaja del ensemble: al promediar cientos de árboles entrenados con bootstrap y feature randomness, el bosque reduce drásticamente la varianza individual de cada CART y captura interacciones no lineales que un árbol único con `max_depth=4` no puede expresar.

5.2. Sobre el algoritmo de optimización

El coordinate descent demostró ser una elección adecuada para este problema por tres razones:

1. **Eficiencia:** redujo de ~ 240 a ~ 16 evaluaciones por pasada respecto a un grid exhaustivo.
2. **Interpretabilidad:** cada iteración produce una decisión auditable (“`max_depth` subió de 15 a None, F1 pasó de 0,589 a 0,602”), facilitando el reporte didáctico.
3. **Validez matemática sobre espacios discretos:** no requiere gradiente, por lo que opera correctamente sobre variables enteras y categóricas.

La principal limitación es la potencial caída en un óptimo local: el algoritmo podría no detectar mejoras que requieran cambiar dos coordenadas simultáneamente. Como trabajo futuro se podría comparar con algoritmos evolutivos o con optimización bayesiana (`Optuna`, `Hyperopt`) que muestrean el espacio en conjunto.

5.3. Sobre el ajuste del umbral para estrategia bancaria

El análisis empírico de la sección 4.5 mostró que el umbral τ^* que maximiza F1 sobre el test (0,516) coincide casi exactamente con el default (0,5). La razón es que `class_weight='balanced'`

ya corrigió el desequilibrio durante el entrenamiento: el bosque internaliza el costo de los errores de la clase minoritaria en cada split, dejando la frontera de decisión bien centrada.

Sin embargo, **el F1 no necesariamente refleja el criterio comercial**. La decisión operativa depende del trade-off:

- **Falsos positivos:** llamadas innecesarias a clientes no interesados ($\Rightarrow$ desgaste de relación + costo operativo).
- **Falsos negativos:** clientes propensos a suscribir que no se contactan ($\Rightarrow$ ingresos perdidos).

Si los FN tienen un costo monetario significativamente mayor que los FP — escenario habitual en una campaña dirigida donde la suscripción genera ingresos recurrentes —, el banco podría operar con umbrales menores (0,15–0,30), aceptando una caída en precisión a cambio de recall más alto. La curva F1 vs umbral generada por el notebook permite trazar este trade-off explícitamente, y `predecir_cliente(..., umbral= $\tau$ )` acepta el parámetro para que la operación decida el punto de corte de acuerdo con su capacidad de llamadas y la economía de la campaña.

5.4. Sobre las variables más predictivas

Que `duration` domine la importancia coincide con el hallazgo seminal de Moro et al. [3]: llamadas más largas indican un cliente realmente interesado, pero también plantea una **advertencia operativa**: la duración solo se conoce *después* de la llamada, por lo que un modelo que dependa fuertemente de ella es más útil para análisis retrospectivo que para predicción a priori. Una línea futura es entrenar un modelo restringido (sin `duration`) para uso en tiempo real de pre-segmentación.

5.5. Conclusiones

1. El Random Forest, entrenado mediante un pipeline *libre de data leakage* y optimizado por coordinate descent sobre F1 estratificado, alcanzó una mejora del $\sim 28,8\%$ sobre el árbol baseline en validación cruzada, con un F1 de 0,62 sobre el test holdout.
2. La curva OOB se reveló como un mecanismo de selección de $n_{\text{estimators}}$ matemáticamente fundamentado y barato, evitando un grid externo.
3. Coordinate descent es válido sobre espacios discretos donde gradient descent no aplica, y entrega configuraciones interpretables con menor costo computacional que un grid search.
4. El uso explícito de `class_weight='balanced'` y la elección de F1 como objetivo son requisitos no opcionales en datasets desbalanceados como Bank Marketing.
5. El pipeline persistido permite predicción on-demand de clientes nuevos, alineándose con la integración real en un sistema de campañas bancarias.

Referencias

- [1] Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth International Group.
- [2] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [3] Moro, S., Cortez, P., & Rita, P. (2014). A data-driven approach to predict the success of bank telemarketing. *Decision Support Systems*, 62, 22–31.
- [4] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.

- [5] Wright, S. J. (2015). Coordinate descent algorithms. *Mathematical Programming*, 151(1), 3–34.
- [6] Probst, P., Wright, M. N., & Boulesteix, A. L. (2019). Hyperparameters and tuning strategies for random forest. *WIREs Data Mining and Knowledge Discovery*, 9(3), e1301.
- [7] Moro, S., Rita, P., & Cortez, P. (2014). Bank Marketing [Dataset]. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/222/bank+marketing>.